



Extending Harbinger – Signal Preprocessing & Anomaly Detection

Roadmap

Harbinger is more than a catalog of built-in methods. It is a framework with clear contracts for transformations, detectors, motif flows, and evaluators. This slide series introduces each extension point progressively.

01

Transformation

Custom median filter

03

Sequence Anomaly

Collective detector with intervals

05

Motif

DTW subsequence clustering

02

Anomaly Detector

Wrapper for `forecast::tsoutliers()`

04

Change Point

Joinpoint regression with `segmented`

06

Evaluation

Custom range-aware evaluator

Signal Preprocessing: Why Filter Before Detecting?

Raw time series often contain isolated spikes — sudden, short-lived deviations caused by sensor noise, transmission errors, or measurement artifacts. If fed directly into a detector, these spikes can trigger false positives or mask true anomalies. A preprocessing transformation cleans the signal before detection, improving both precision and interpretability.

Isolated Spike

A single point that deviates sharply from its neighbors. Not a true anomaly — just noise. A median filter replaces it with the local median, preserving the surrounding trend.

Median Filter (runmed)

A non-linear smoother: for each point, it computes the median of a sliding window of size k . Unlike a mean filter, it is robust to outliers and does not blur edges.

Transformation Contract

In Harbinger, any transformation only needs two things: a constructor (`dal_transform()`) and a `transform()` method. The rest of the pipeline — detection, evaluation, plotting — remains unchanged.

Further reading: Ogasawara, E. et al. (2025). Event Detection in Time Series. Springer. — Chapter on preprocessing and transformation pipelines.

Custom Transformation: Median Filter

Goal: apply `stats::runmed()` to the series before the detector, smoothing isolated spikes without distorting trends.

```
library(daltoolbox)
library(harbinger)

har_fil_median_custom <- function(k = 5) {
  if (k %% 2 == 0) k <- k + 1
  obj <- daltoolbox::dal_transform()
  obj$k <- k
  class(obj) <- append(
    "har_fil_median_custom", class(obj))
  obj
}
```

```
transform.har_fil_median_custom <-
function(obj, data, ...) {
  result <- stats::runmed(
    as.numeric(data),
    k = obj$k,
    endrule = "keep")
  result[is.na(result)] <-
    as.numeric(data)[is.na(result)]
  result
}

# Use
filter_custom <- har_fil_median_custom(k =
5)
serie_filtered <- transform(
  filter_custom, dataset$serie)
```

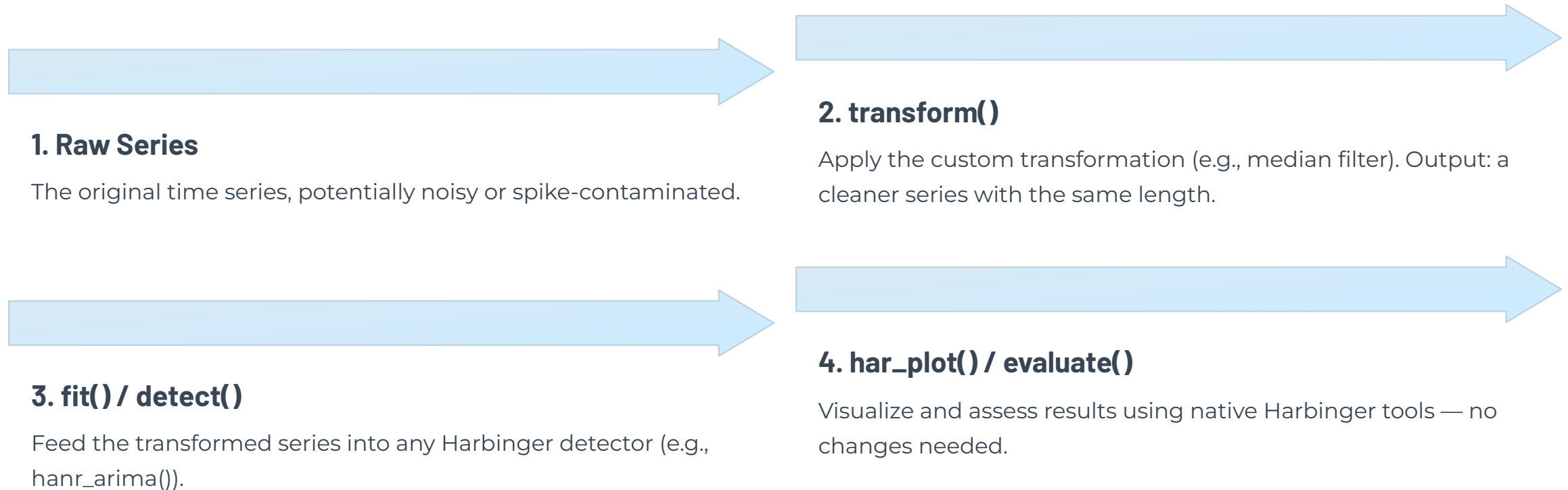
Step by Step

- Ensure `k` is odd (required by `runmed`)
- Create base object with `dal_transform()`
- Append custom class with `append()`
- Implement `transform()`: apply `runmed()` and preserve NAs
- Use the object exactly like any native transformation

 **Key concept:** A custom transformation only needs a constructor and a `transform()` method. The rest of the workflow remains unchanged.

The Transformation → Detection Pipeline

A transformation is not a detector — it is a signal conditioning step. In Harbinger, transformations and detectors are composable: you apply one or more transformations to the raw series, then pass the result to any detector. This separation of concerns keeps each component focused and reusable.



Further reading: Ogasawara, E. et al. (2025). Event Detection in Time Series. Springer. — Chapter on the Harbinger extension architecture.

Applying the Transformation in the Pipeline

After filtering the series, the ARIMA detector is fed the smoothed signal. The median filter is a pre-processing step, not a detector.

```
data(examples_anomalies)
dataset <- examples_anomalies$simple

# Visualizar série bruta
har_plot(harbinger(),
  dataset$serie,
  event = dataset$event)

# Aplicar filtro e visualizar
filter_custom <- har_fil_median_custom(k =
5)
serie_filtered <- transform(
  filter_custom, dataset$serie)

har_plot(harbinger(),
  serie_filtered,
  event = dataset$event)
```

```
# Detectar com ARIMA sobre série filtrada
model <- hanr_arima()
model <- fit(model,
  as.numeric(serie_filtered))
detection <- detect(model,
  as.numeric(serie_filtered))

har_plot(model,
  as.numeric(serie_filtered),
  detection,
  dataset$event)
```

Step by Step

- Load data with `examples_anomalies$simple`
- Plot the raw series as a visual reference
- Apply `transform()` to obtain the filtered series
- Plot the filtered series and compare with the raw one
- Fit `hanr_arima()` on the filtered series
- Detect and plot the final result

📌 **Key concept:** Transformation → Detection. Changing the signal representation first improves the interpretability of the downstream detector.

Pointwise Anomaly Detection: Concepts & Integration

A pointwise anomaly is an individual observation that deviates significantly from the expected behavior of the series. Detecting them requires a model of normality — a baseline against which each point is compared. Harbinger's open architecture allows any external detection library to be wrapped as a native detector, as long as it respects a minimal two-part contract.

Pointwise Anomaly

A single timestamp where the observed value is statistically inconsistent with the surrounding context. Common causes: sensor faults, data entry errors, sudden real-world events.

`tsoutliers()` (forecast pkg)

Detects additive outliers (AO) and temporary changes (TC) by fitting an ARIMA model and identifying residuals that exceed a threshold. Returns the index and replacement value for each outlier.

Detector Contract

In Harbinger, a custom detector needs only: (1) a constructor based on `harbinger()`, and (2) a `detect()` method that returns a standardized detection table via `har_restore_refs()`. No reimplementation of plotting or evaluation is needed.

Further reading: Ogasawara, E. et al. (2025). Event Detection in Time Series. Springer. — Chapter on pointwise anomaly detectors and the Harbinger detector contract.

Custom Detector: `forecast::tsoutliers()`

Goal: wrap `forecast::tsoutliers()` as an anomaly detector in Harbinger, identifying additive outliers and temporary disturbances.

```
library(daltoolbox)
library(harbinger)
# install.packages("forecast")

hanr_tsoutliers_custom <-
  function(frequency = 1) {
    obj <- harbinger()
    obj$frequency <- frequency
    class(obj) <- append(
      "hanr_tsoutliers_custom", class(obj))
    obj
  }
```

```
detect.hanr_tsoutliers_custom <-
  function(obj, serie, ...) {
    obj <- obj$har_store_refs(obj, serie)

    y_ts <- stats::ts(obj$serie,
                      frequency = obj$frequency)
    out <- forecast::tsoutliers(y_ts)

    anomalies <- rep(FALSE, length(obj$serie))
    if (!is.null(out$index) &&
        length(out$index) > 0) {
      anomalies[unique(out$index)] <- TRUE
    }

    obj$har_restore_refs(obj,
                        anomalies = anomalies)
  }
```

Step by Step

- Create constructor based on `harbinger()`
- Store frequency in the object
- Implement `detect()`: convert series to `ts`
- Call `forecast::tsoutliers()` and capture indices
- Map positions to a logical anomaly vector
- Return with `har_restore_refs()`

📌 **Key concept:** The integration contract is minimal — constructor + `detect()` method. The external library decides the suspicious timestamps; Harbinger standardizes the result.

Evaluating Anomaly Detectors: The Confusion Matrix

Once a detector produces its output, the next step is evaluation. Harbinger uses a standard confusion matrix to compare detected events against the ground truth. Understanding the four cells — TP, FP, FN, TN — is essential to interpreting detector performance and choosing the right trade-off between sensitivity and precision.

True Positive (TP)

A real anomaly that was correctly detected. The detector fired at the right moment.

False Positive (FP)

A normal point incorrectly flagged as anomalous. Indicates over-sensitivity or noise in the detector.

False Negative (FN)

A real anomaly that was missed. The detector failed to fire when it should have.

True Negative (TN)

A normal point correctly left unflagged. The majority of points in most series.

In Harbinger, `evaluate(model, detection$event, dataset$event)` computes the confusion matrix automatically. A perfect detector yields 0 FP and 0 FN.

Further reading: Ogasawara, E. et al. (2025). Event Detection in Time Series. Springer. — Chapter on performance metrics for event detection.

Running and Evaluating the Custom Detector

Once the detector is defined, its usage is identical to any native Harbinger detector: `detect()`, `evaluate()`, and `har_plot()`.

```
data(examples_anomalies)
dataset <- examples_anomalies$simple

model <- hanr_tsoutliers_custom()
detection <- detect(model, dataset$serie)

# Avaliar
evaluation <- evaluate(model,
                        detection$event,
                        dataset$event)
evaluation$confMatrix
```

```
## event
## detection TRUE FALSE
##   TRUE  0   0
##   FALSE 1 100

# Visualizar
har_plot(model,
          dataset$serie,
          detection,
          dataset$event)
```

Step by Step

- Instantiate with `hanr_tsoutliers_custom()`
- Call `detect(model, serie)` — no `fit()` required
- Evaluate with `evaluate()` and inspect `confMatrix`
- Plot with `har_plot()` for visual inspection

📌 **Key concept:** A custom detector does not need to re-implement the full workflow. Respecting the constructor + `detect()` contract is enough to reuse native plots and evaluation.

Summary of Extension Points

Each extension respects a minimal contract. The smaller the contract, the easier it is to integrate external libraries into the Harbinger workflow.

Transformation

Contract: constructor + `transform()`

Example: `har_fil_median_custom` with `stats::runmed()` — smooths isolated spikes before detection without distorting trends.

Anomaly Detector

Contract: constructor + `detect()`

Example: `hanr_tsoutliers_custom` with `forecast::tsoutliers()` — identifies additive outliers and temporary disturbances.

Additional Sources

Explore the resources below to dive deeper into Harbinger and time series event detection.



CRAN Package

cran.r-project.org/web/packages/harbinger



Event Detection Resources

eic.cefet-rj.br/~eogasawara/en/event-detection-in-time-series



DAL Toolbox

cran.r-project.org/web/packages/daltoolbox



Issues & Discussions

github.com/cefet-rj-dal/harbinger/issues

Open questions, report bugs, or suggest improvements directly in the repository.

References

Ogasawara, E. et al. (2025). Event Detection in Time Series. Springer.

Primary reference for the Harbinger framework.

Hyndman, R.J., & Khandakar, Y. (2008). Automatic time series forecasting: the forecast package for R. *Journal of Statistical Software*, 27(3), 1–22. — Relevant for `forecast::tsoutliers()`.

Cleveland, R.B., Cleveland, W.S., McRae, J.E., & Terpenning, I. (1990). STL: A seasonal-trend decomposition procedure based on loess. *Journal of Official Statistics*, 6(1), 3–73. — Relevant for robust smoothing and median-based filtering.

Muñoz-Cobo, J.L. et al. (2017). Use of the Kolmogorov–Smirnov test to check for outliers in time series. *Energies*, 10(12), 1934. — Relevant for outlier detection methods.

Bischi, B. et al. (2012). Resampling methods for time series. — Relevant for cross-validation in time series contexts.